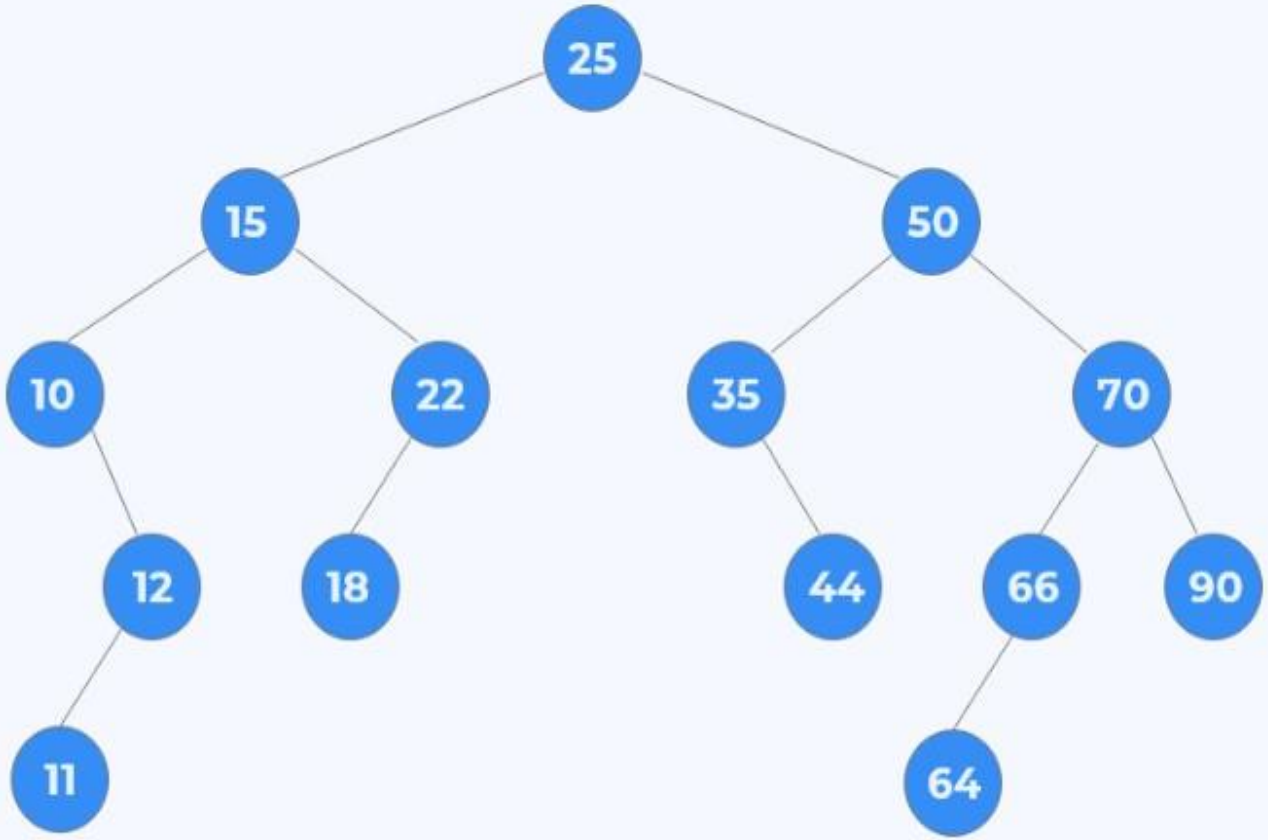
Designing and Analysis of Algorithms

Course Code: ECS 5101/CS514

- Dr Rahul Mishra
- IIT Patna

- **Lecture 14**



PreOrder:

25 15 10 12 11 22 18 50 35 44 70 66 64 90

PostOrder:

11 12 10 18 22 15 44 35 64 66 90 70 50 25

Inorder:

10 11 12 15 18 22 25 35 44 50 64 66 70 90

BINARY SEARCH TREES :-

→ One of the most important data structures in computer science.

→ This structure enables one to search for and find an element with an average running time $f(n) = O(\log_2 n)$.

→ This contrast with :

1) Sorted linear array (binary search) $O(\log_2 n)$
2) Linear linked list $(O(n))$

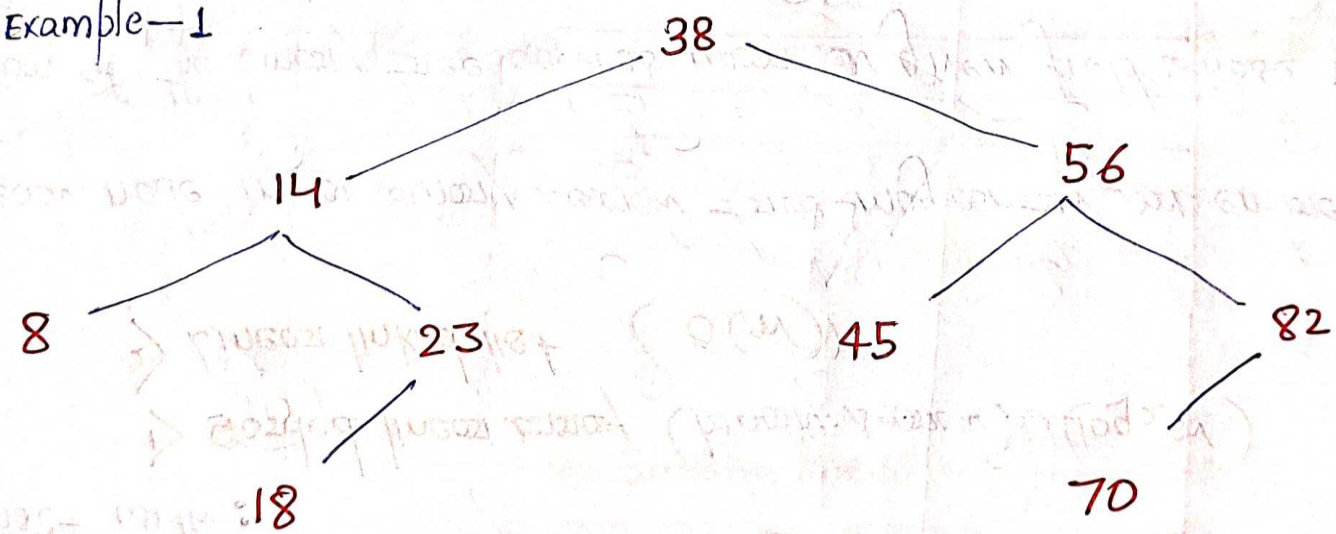
→ Although each node in a binary search tree may contain an entire record of data.

→ The definition of the binary tree depends upon a given field whose values are distinct any may be ordered.

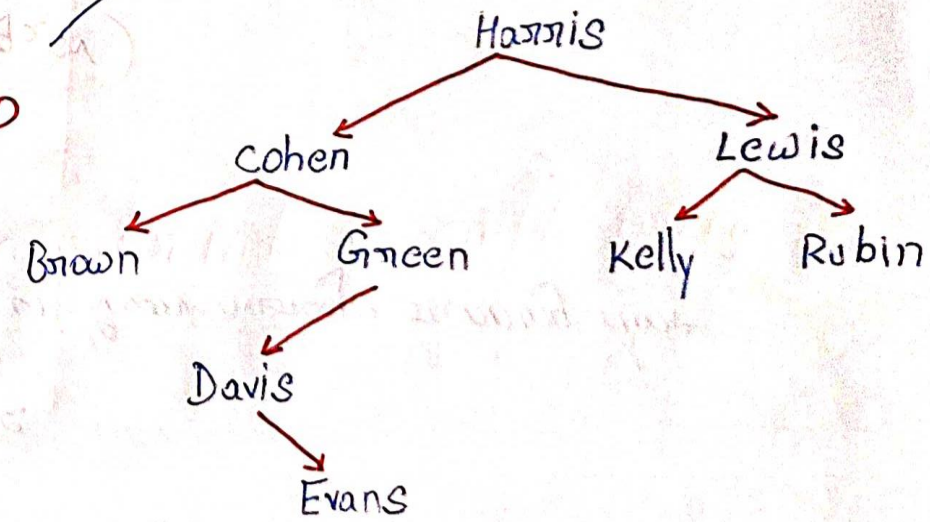
A binary tree T is called binary search tree (or binary sorted tree) if each node N of T has the following property:

The value at (N) is greater than every value in the left subtree of (N) and is less than every node value in the right subtree.

Example–1

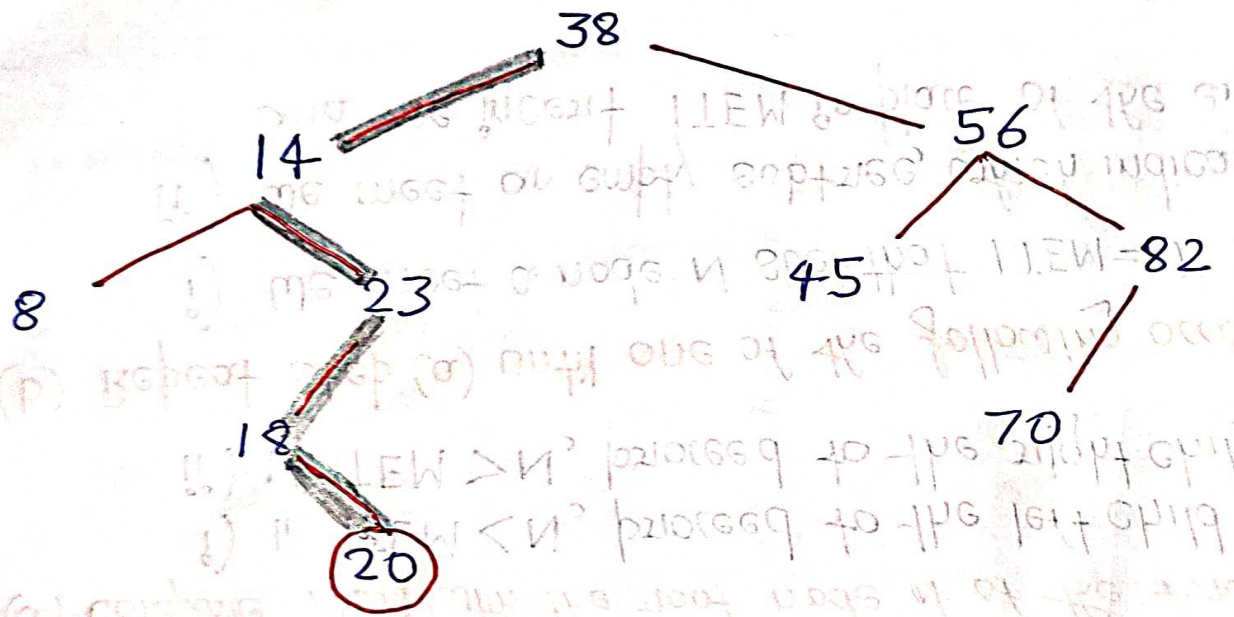


Example–2



Example →

Instead (20) in the previous binary search tree :-



Class Assignment :- 40, 60, 50, 33, 55, 11

Suppose the following six numbers are inserted in order into an empty binary search tree.

* Complexity of the Searching Algorithm:

> The number of comparisons in **BST** is bounded by the depth of the tree.

> This comes from the fact that we proceed down a single path of the tree.

> The running time of the search will be proportional to the depth of the tree.

> We are given n data items, $A_1, A_2, \ldots, A_N$, and suppose the items are inserted in order into a binary search tree T.

> $f(n) = O(\log_2 n)$.

* Application of Binary Search Trees:-

> Deleting duplicate node from a list of number →

Approach: A Scan the elements from A_1 to A_N (that is, from left to right)

(a) For each element A_K compare A_K with $A_1, A_2, \ldots, A_{K-1}$, that is, compare A_K with those elements which precede A_K.

(b) IF A_K does occur among $A_1, A_2 \ldots A_{K-1}$ then delete A_K

Deleting In a Binary Search Tree :-

> The deletion algorithm first uses "FIND Algorithm" (discussed previously) to find the location of the node N which contains ITEM and also the location of the parent node (PN)

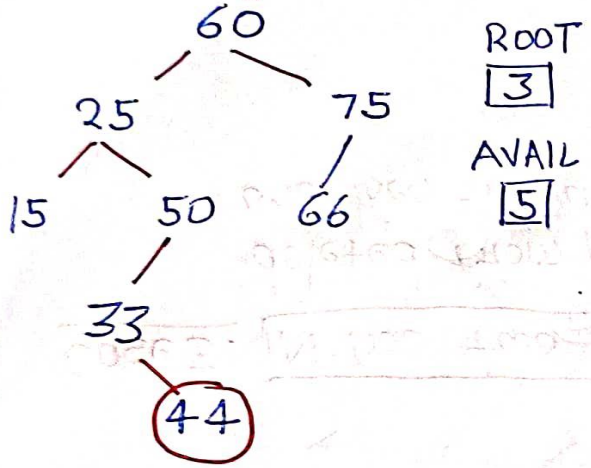
> There are three cases of deletion:

Case 1. N has no children. Then N is deleted from T by simply replacing the location of N in the parent node P(N) by the null pointer.

Case 2. N has exactly one child. Then N is deleted from T by simply replacing the location of N in P(N) by the location of the only child of N.

Case 3. N has two children. Let S(N) denote the "inorder successor of N." Then N is deleted from T by first deleting S(N) from T (by using Case 1 or Case 2) and then replacing node N in T by the node S(N).

* Before deletion



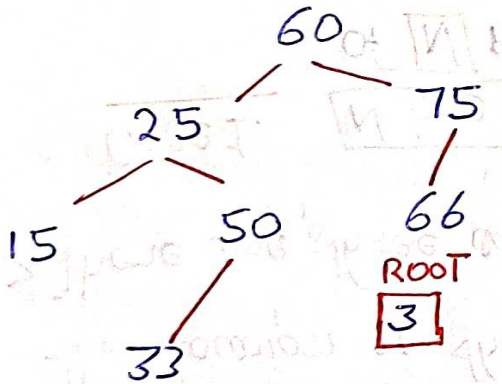
ROOT [3]

AVAIL [5]

Before deletion

	INFO	LEFT	RIGHT
1	33	0	9
2	25	8	10
3	60	2	7
4	66	0	0
5		6	
6		0	
7	75	4	0
8	15	0	0
9	44	0	0
10	50	1	0

a) We delete node 44 ⟶

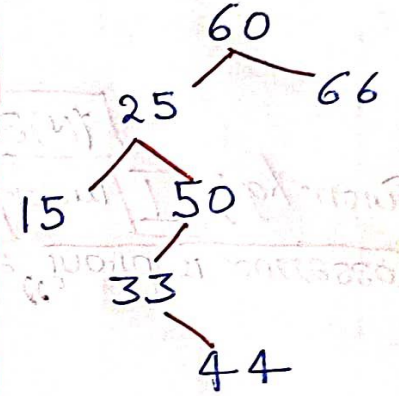


ROOT [3]

AVAIL [9]

	INFO	LEFT	RIGHT
1	33	0	9
2	25	8	10
3	60	2	4
4	66	0	0
5		6	
6		0	
7		5	
8	15	0	0
9	44	0	0
10	50	1	0

b) Node 75 is deleted ⟶



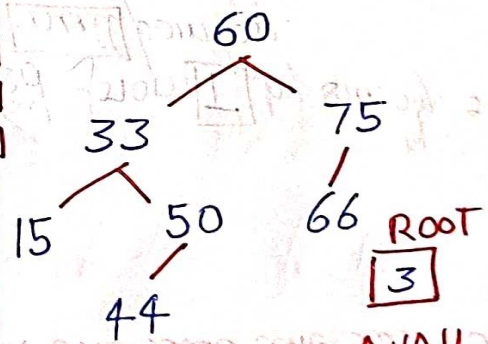
ROOT [3]

AVAIL [7]

	INFO	LEFT	RIGHT
1	33	0	9
2	25	8	10
3	60	2	4
4	66	0	0
5		6	
6		0	
7		5	
8	15	0	0
9	44	0	0
10	50	1	0

c) Node 25 deleted

Note:— 33 is the Inorder sucessor of 25

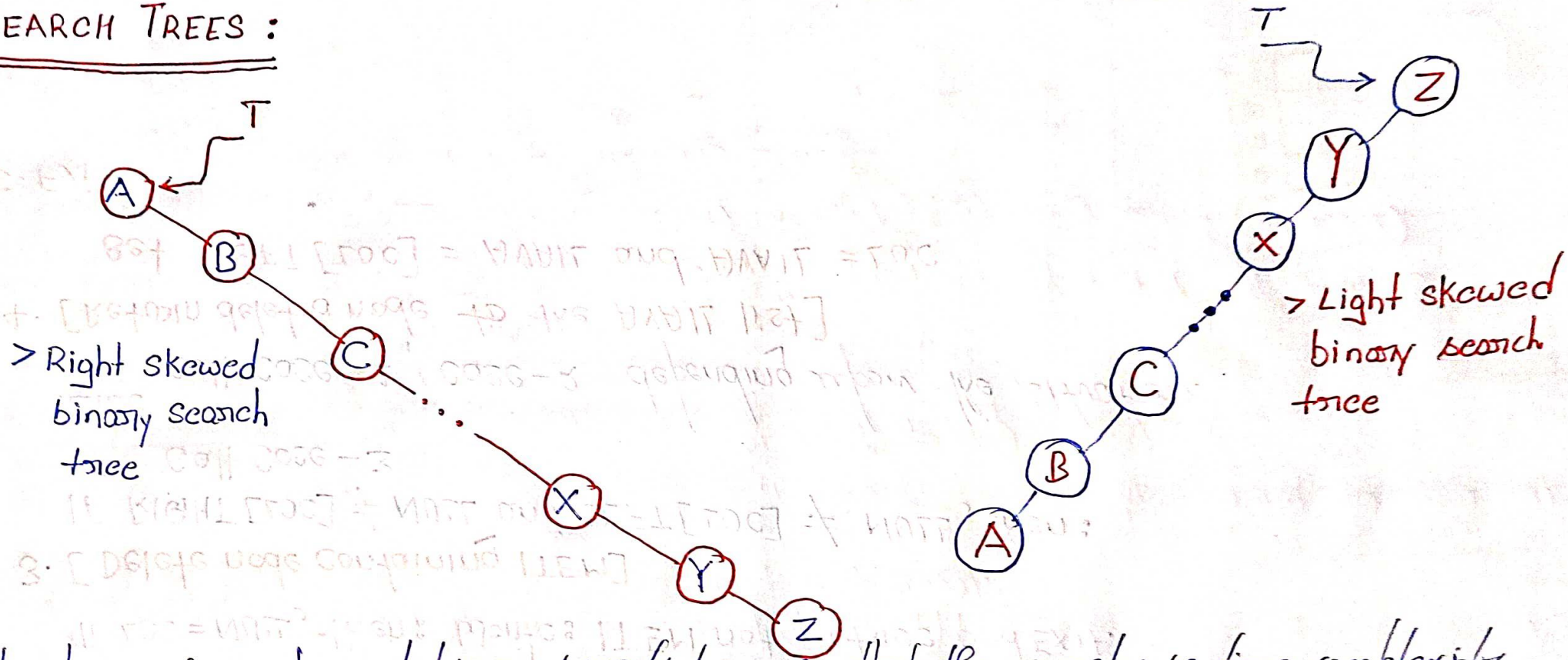


ROOT [3]

AVAIL [2]

	INFO	LEFT	RIGHT
1	33	8	10
2		5	
3	60	1	7
4	66	0	0
5		6	
6		0	
7	75	4	0
8	15	0	0
9	44	0	0
10	50	9	0

AVL SEARCH TREES :



> The disadvantage of a skewed binary search tree is that the worst case time complexity of a search is O(n).

> These arises the need to maintain the binary search tree to be of balanced height.

> By doing so it is possible to obtain for the search operation a time complexity of O(log n) in the worst case.

> One of the more popular balanced trees was by Adelson - Velskii and Landis (AVL)

Definition :-

→ An empty binary tree is an AVL tree.

→ Given T^L and T^R to be the left and right subtrees of T and $h(T^L)$ and $h(T^R)$ to be the heights of subtrees T^L and T^R, respectively.

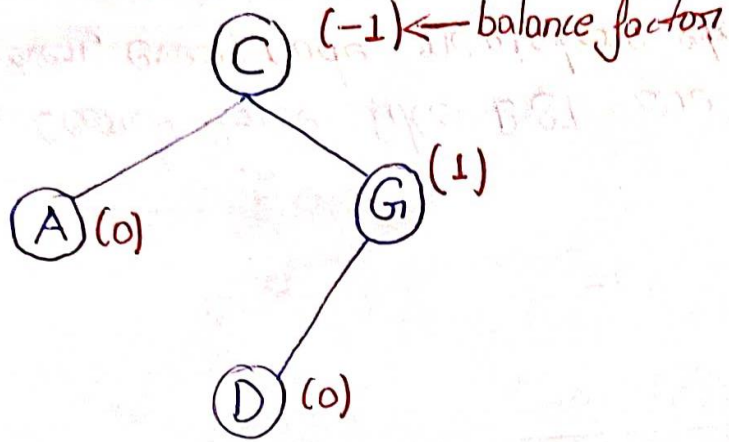
→ T^L and T^R are AVL trees and $|h(T^L) - h(T^R)| \leq 1$

Balance factor (BF)

↳ For an AVL tree the balance factor of a node can be either 0, 1, or -1.

* AVL search tree is a binary search tree which is an AVL tree.

→ AVL search tree like BST are represented using a linked representation However, every node registers its balance factor.



Searching an AVL Search Tree →

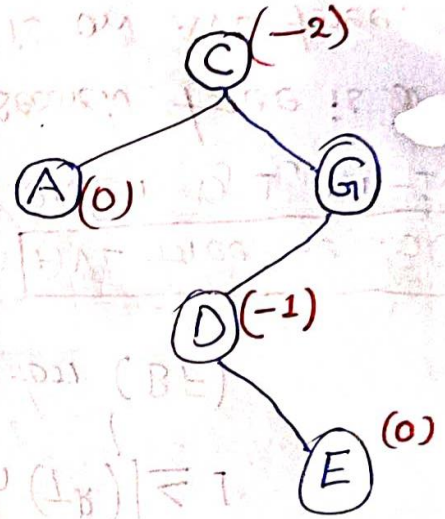
* Searching an AVL search tree for an element is exactly similar to the method used in BST.

* Insertion in an AVL Search tree :-

> Inserting an element into an AVL search tree in its first phase is similar to that of the one used in BST.

> If after insertion of the element, the balance factor of any node in the tree is affected
→ renders the BST unbalanced

↳ If we insert E in the previous tree.



> To overcome this situation an important technique is introduced namely Rotations.

→ To perform rotations it is necessary to identify a specific node A whose BF(A) is neither 0, 1, or -1.

→ It is nearest ancestor to the inserted node on the path from inserted node to the root.

→ The rebalancing rotations are classified as LL, LR, RR, and RL.

LL rotation: Inserted node is the left subtree of left subtree of node A.

RR rotation: Inserted node is the right subtree of right subtree of node A.

LR rotation: Inserted node is the right subtree of left subtree of node A

RL rotation: Inserted node is the left subtree of right subtree of node A.